Bio-Inspired Locomotion for *Walter* (Walking Alternating Tripod for Evolutionary Research): CAD, Firmware, Optimization, and MuJoCo PPO

Howard Wang and Matthew Shiley

Engineering 90
Advisor(s): Allan Moser

April 14, 2026

Abstract

Walter (**W**alking **A**lternating **T**ripod for **E**volutionary **R**esearch) is a twelve-degree-of-freedom hexapod built for this Engineering 90 capstone. The team fabricated a **complete hardware platform**: CAD-designed, 3D-printed body, legs, and linkages; hobby servos; Teensy 4.1 and Pololu eighteen-channel driver; SparkFun BMI270 IMU; power regulation; and three iterations of custom *Hexapod V2* PCBs. On the control side, the project **completed two development tracks**: (i) open-loop gait optimization in a reduced-order model with deterministic export to firmware (`hexapod_openloop_sine`), and (ii) a MuJoCo [1] PPO [2] training and evaluation pipeline (Stable-Baselines3 [3]) with stored checkpoints, configs, and short evaluation summaries for representative locomotion experiments. Sec. 9 summarizes surrogate metrics, exported gait parameters, a MuJoCo model view, and sample PPO evaluation means. The team **did not** implement the full layered adaptive firmware described in internal phase requirements (IMU-closed-loop reflexes and online adaptation on the MCU); that stack remains **design groundwork** for future work. Overall, the capstone establishes a **reproducible hexapod platform** and defensible optimization-to-firmware and simulation-based RL workflows; materials are at `https://github.com/HowardWHSrun/WALTER`. **Purchased cost:** electronics (three BOMs) US$595.20; three PCB orders US$197.47; **grand total US$792.67** (one PCB order awaited payment at writing). Filament and fasteners are outside this sum.

1 Introduction

Reliable legged locomotion requires coordinating periodic limb motion with feedback under limited sensing and computation. Insects organize locomotor control in *layers* that are standard in the literature: rhythmic coordination (often modeled as CPG-like structure), fast sensory feedback, and slower parameter adjustment [4, 5]. This capstone uses that organization as **bio-inspired control motivation**, not as a claim that every layer is already running on hardware.

Framing of contributions. The strongest completed work is **platform and methods**: a reproducible hexapod with CAD and custom PCBs; an **open-loop** gait optimization loop that exports parameters to firmware; and a **MuJoCo PPO** training and evaluation workflow with stored checkpoints and configs. A **layered adaptive** architecture (IMU fusion, reflexes, slow adaptation) is **specified in design documents** and informs observation choices in simulation, but it is **not fully implemented** on the MCU. The report therefore emphasizes what was built and verified, and treats full adaptive locomotion on the robot as **future engineering**, not an accomplishment of this submission.

Bio-inspired elements (precise). The **alternating tripod** is a biologically motivated low-dimensional gait prior: two stance groups alternate before any learning is applied. **IMU-based body-state cues** (roll, pitch, rates) match how many legged systems use gravito-inertial information for posture [6, 7] and align the simulation observation stack with how *Walter* is instrumented, which matters for eventual sim-to-real. **Reinforcement learning in MuJoCo** explores dynamics and reward shaping under that prior; it does not, by itself, close the hardware adaptive loop. **Surrogate optimization** of a compact sinusoidal law isolates an interpretable rhythm before sensor-heavy layers are added—consistent with separating pattern generation from feedback in many biological models [5], but realized here only through open-loop firmware.

Platform. The physical system is *Walter*, a twelve-DOF hexapod with 3D-printed structure, servos, Teensy, driver, IMU, and *Hexapod V2* PCBs (Sec. 6.5–6.6).

Code and report organization. Source code and this document are maintained at `https://github.com/HowardWHSrun/WALTER`. Sec. 2 states the optimization objective and the PPO machinery used in software. Sec. 3–5 and Table 1 summarize three tracks: **(1)** surrogate optimization to firmware (**completed**); **(2)** PPO locomotion in simulation, with IMU-capable observation modes where configured (**completed** as a training/evaluation stack; limited hardware deployment); **(3)** vision-augmented distance objectives (**future work**).

2 Mathematical background: optimization and reinforcement learning

2.1 Reduced-order gait optimization (Track 1)

2.1.1 Decision variables and bounds

The optimizer acts on a vector $\boldsymbol{\phi} \in \mathbb{R}^{25}$ matching `hexapod_no_imu_optimization.py`. Indices are concatenated as

$$\boldsymbol{\phi} = \left(\boldsymbol{\theta}_{\mathrm{abd,rest}}^{\top}, \boldsymbol{\theta}_{\mathrm{flx,rest}}^{\top}, \boldsymbol{\delta}_{\mathrm{abd}}^{\top}, \boldsymbol{\delta}_{\mathrm{flx}}^{\top}, f\right)^{\top}, \tag{1}$$

where each of $\boldsymbol{\theta}_{\mathrm{abd,rest}}, \boldsymbol{\theta}_{\mathrm{flx,rest}}, \boldsymbol{\delta}_{\mathrm{abd}}, \boldsymbol{\delta}_{\mathrm{flx}} \in \mathbb{R}^6$ are in *degrees* in the code (converted to radians internally), indexed by leg order FL, FR, ML, MR, RL, RR, and $f \in \mathbb{R}_+$ is the common gait frequency in hertz. Box constraints are implemented as `scipy.optimize.Bounds`: for example $f \in [0.4,\ 4.0]\,\mathrm{Hz}$; resting abduction per joint in $[-25°, 25°]$; resting flexion in $[10°, 65°]$; phase shifts in $[-120°, 120°]$. These define a feasible orthotope $\boldsymbol{\phi}_{\min} \leq \boldsymbol{\phi} \leq \boldsymbol{\phi}_{\max}$.

2.1.2 Open-loop joint commands (sinusoidal law)

Let $\omega = 2\pi f$. For leg $\ell \in \{1, \ldots, 6\}$ with fixed tripod offset $\psi_\ell \in \{0, \pi\}$ (alternating tripod), time $t \geq 0$, and fixed amplitude tables $\bar{a}_{\mathrm{abd},\ell}$, $\bar{a}_{\mathrm{flx},\ell}$ (defaults in code, not decision variables), the commanded abduction and flexion angles are

$$\phi_{\mathrm{abd},\ell}(t) = \theta_{\mathrm{abd,rest},\ell} + \bar{a}_{\mathrm{abd},\ell} \sin\!\big(\omega t + \psi_\ell + \delta_{\mathrm{abd},\ell}\big), \tag{2}$$

$$\phi_{\mathrm{flx},\ell}(t) = \theta_{\mathrm{flx,rest},\ell} + \bar{a}_{\mathrm{flx},\ell} \sin\!\big(\omega t + \psi_\ell + \delta_{\mathrm{flx},\ell}\big) - \Delta_{\mathrm{lift},\ell}(t), \tag{3}$$

where $\Delta_{\mathrm{lift},\ell}$ implements swing-phase foot clearance (subtracting flexion when the abduction sine is positive), identical in structure to the firmware `LIFT_SWING_DEG_PEAK` term. Velocities $\dot{\phi}_{\mathrm{abd},\ell}$, $\dot{\phi}_{\mathrm{flx},\ell}$ are obtained by differentiation and feed the planar surrogate dynamics.

2.1.3 Surrogate planar dynamics and simulation

The reduced model integrates an eight-dimensional state

$$\mathbf{x}(t) = \big(x,\ \dot{x},\ z,\ \dot{z},\ \phi_{\mathrm{roll}},\ \dot{\phi}_{\mathrm{roll}},\ \phi_{\mathrm{pitch}},\ \dot{\phi}_{\mathrm{pitch}}\big)^\top, \tag{4}$$

with $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \boldsymbol{\phi})$ assembled from lumped masses, soft ground contact, leg thrust and support forces derived from $\phi_{\mathrm{abd},\ell}$, $\phi_{\mathrm{flx},\ell}$ and contact weights, and linear damping on body motion and attitude rates (`body_ode` in the script). For fixed $\boldsymbol{\phi}$, the trajectory is obtained by `solve_ivp` (Runge–Kutta) over $t \in [0, T]$ with T the simulation horizon (`sim_duration_s`), sampled at N time points (`sample_count`).

2.1.4 Scalar objective (explicit weighted sum)

Let a subscript "ss" denote statistics evaluated on a *steady-state* suffix of the horizon (the code uses the last 60% of samples for oscillation metrics and mean speed). Write $\bar{v}_x$ for mean forward velocity $\mathbb{E}_{\mathrm{ss}}[\dot{x}]$, and RMS values $\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{roll}}]$, $\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{pitch}}]$, $\mathrm{RMS}_{\mathrm{ss}}[z]$. Let $q_{\mathrm{servo}} \geq 0$ be a slack on peak torque proxy exceeding a threshold (0.85 of stall in the implementation), and $\sigma_{\mathrm{contact}}$ the time mean of the per-timestep standard deviation of contact weights across legs. With nonnegative weights w. matching `OptimizationConfig`, the implemented cost is

$$\begin{aligned}\mathcal{J}(\boldsymbol{\phi}) = -w_{\mathrm{fwd}}\,\bar{v}_x + w_{\mathrm{back}}\left[\max(0,-\bar{v}_x)\right]^2 \\ + w_{\mathrm{rp}}\left(\mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{roll}}]^2 + \mathrm{RMS}_{\mathrm{ss}}[\phi_{\mathrm{pitch}}]^2\right) + w_z\,\mathrm{RMS}_{\mathrm{ss}}[z]^2 \\ + w_{\mathrm{servo}}\,q_{\mathrm{servo}}^2 + w_{\mathrm{contact}}\,\sigma_{\mathrm{contact}}^2 + w_{\mathrm{reg}}\,\|\boldsymbol{\phi}-\boldsymbol{\phi}_0\|_{\mathrm{scaled}}^2,\end{aligned} \tag{5}$$

where $\boldsymbol{\phi}_0$ is the default parameter vector and $\|\cdot\|_{\mathrm{scaled}}$ averages coordinatewise squared normalized deviation from the box midpoint (Tikhonov-style regularization keeping $\boldsymbol{\phi}$ in a "reasonable" neighborhood). Optional terms penalize shortfall below a target mean speed if `target_forward_speed_mps` $>$ 0. Infeasible simulations return a large penalty (10^6). **Design intent:** the weighted sum prioritizes forward progress while penalizing backward drift, attitude and vertical oscillation, actuator overload proxies, uneven leg loading, and deviation from nominal $\boldsymbol{\phi}_0$—a deliberate trade that can admit energetic gaits with visible body motion in the surrogate.

2.1.5 Optimization algorithm and restarts

The problem solved in software is

$$\min_{\boldsymbol{\phi}} \; \mathcal{J}(\boldsymbol{\phi}) \quad \text{subject to} \quad \boldsymbol{\phi}_{\min} \le \boldsymbol{\phi} \le \boldsymbol{\phi}_{\max}. \tag{6}$$

L-BFGS-B (`scipy.optimize.minimize`, method `L-BFGS-B`) approximates curvature of $\mathcal{J}$ with a limited-memory inverse Hessian and projects each iterate onto the box. Each evaluation of $\mathcal{J}$ requires one numerical integration of the surrogate. The outer loop runs R random restarts (`restarts`): one start at $\boldsymbol{\phi}_0$ and $R-1$ uniform samples in the box; the best $\boldsymbol{\phi}^\star$ by lowest $\mathcal{J}$ is retained. Limits `maxiter`, `maxfun`, and `ftol` cap work per restart.

The **code path** from optimization to firmware is deterministic: $\boldsymbol{\phi}^\star$ is serialized to `no_imu_optimization_summary.` `sync_gait_params_from_json.py` emits `optimized_gait_params.h`; `hexapod_openloop_sine.ino` evaluates the same `leg_commands()` map, applies bench degree-to-pulse gains and clamps, and streams servo targets. Thus optimization-to-code is *algebraic parity*, not a learned policy.

2.2 Policy optimization in MuJoCo (PPO with GAE)

Legged RL is cast as an MDP with MuJoCo supplying transitions; we use actor–critic PPO as implemented in Stable-Baselines3 [3]. **Generalized advantage estimation** (GAE) [2] builds advantages from TD residuals $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$:

$$\hat{A}_t = \sum_{\ell=0}^{\infty} (\gamma\lambda)^\ell\, \delta_{t+\ell}, \tag{7}$$

with λ and discount γ set in YAML. **PPO** [2] stabilizes updates via a clipped ratio between the new and behavior policy. Define the **probability ratio**

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{8}$$

For continuous actions, π_θ is typically Gaussian; r_t is the ratio of densities. The **clipped surrogate objective** for a batch of timesteps is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t\big)\Big], \tag{9}$$

with clip range $\epsilon \in (0,1)$ (often 0.2). Large policy steps that inflate r_t when $\hat{A}_t > 0$ (or shrink r_t when $\hat{A}_t < 0$) are clipped so the effective objective stops increasing, reducing catastrophic updates common in vanilla policy gradients.

Let $\hat{R}_t$ denote a target return for the critic (e.g., n-step return or GAE-backed value target). With coefficients $c_v, c_e > 0$, the **composite loss** maximized in practice is the negative of

$$\mathcal{L}_{\text{total}}(\theta, \psi) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v\,\mathbb{E}_t\big[(V_\psi(s_t) - \hat{R}_t)^2\big] - c_e\,\mathbb{E}_t\big[H(\pi_\theta(\cdot \mid s_t))\big], \tag{10}$$

where H is differential entropy for Gaussian policies. Minimizing $\mathcal{L}_{\text{total}}$ corresponds to ascending $\mathcal{L}^{\text{CLIP}}$ while fitting the critic and discouraging collapse of exploratory variance. Updates use minibatches over several epochs on each rollout buffer; hyperparameters (learning rate, rollout length `n_steps`, batch size, γ, λ, ϵ, c_v, c_e) are set in YAML under `hexapod_training_new/configs/`.

3 Track I: Surrogate optimization and firmware export

This track **implements** Sec. 2.1: the $\boldsymbol{\phi}$ parameterization, sinusoidal `leg_commands()`, the eight-state surrogate, cost (5), box-constrained L-BFGS-B with restarts, and a **deterministic path** from $\boldsymbol{\phi}^\star$ to C headers and `hexapod_openloop_sine`. The workflow isolates a tunable open-loop rhythm before IMU-closed-loop layers are required.

What we shipped. Optimizer script, summary JSON, JSON-to-header sync, and open-loop Arduino sketch with exported gains. Quantitative surrogate outputs appear in Tables 6–7; the high-fidelity MuJoCo asset used for RL appears in Fig. 4.

4 Track II: PPO locomotion in MuJoCo

The policy $\pi_\theta(a|s)$ conditions on **proprioception** (joint positions and velocities), simulator **base kinematics**, and—in IMU-focused YAML configs—**accelerometer and gyroscope** channels (or complementary-filter-style stacks) so roll, pitch, and body rates align with how *Walter* is sensed on hardware. **Tripod structure** enters through phase features, structured policy heads (e.g., sCPG variants), or reward shaping, depending on the experiment. Training uses Eqs. (7)–(9) on MuJoCo rollouts.

Scope. The team **built, trained, and evaluated** policies in simulation, archived checkpoints, and wrote short evaluation summaries for representative runs (Table 8). Deploying a trained policy as the primary closed-loop driver on the physical robot—with the same observation latency and actuation path as firmware—was **out of scope** for verification in this report.

5 Track III: Vision-augmented distance objectives (future)

Future work. Augment s with **visual odometry** or pose from an overhead or onboard camera so rewards can emphasize **distance traveled** with less privileged state. Let $p_t \in \mathbb{R}^2$ be horizontal pose from visual integration or fiducials, and consider forward displacement $x_t - x_0$. A natural reward increment is

$$r_t = \alpha\,(x_{t+1} - x_t) - \beta\,\|\omega_t\|^2 - \rho\,\mathrm{penalty}_{\mathrm{fall}} - \kappa\,\mathrm{TV}(a)_t, \tag{11}$$

with IMU rates ω_t penalizing harsh attitude changes, and optional terms for staying upright. The **observation** becomes $s_t = [s_t^{\mathrm{IMU/proprio}}, f_t]$ where f_t is a vector of visual features or estimated pose; PPO then applies the same surrogate (9) with expanded input dimension.

Open challenges include sim-to-real on vision, defining rewards from estimated versus ground-truth motion, and keeping a tripod prior while learning residuals.

Table 1: Summary of the three development tracks for *Walter*.

Track	Technical core	Outcome in this capstone
I	Surrogate $\mathcal{J}(\boldsymbol{\phi})$ (5); L-BFGS-B; JSON→`.h`→ open-loop firmware	**Completed:** optimization exported and flashed path demonstrated
II	PPO (9); GAE (7); MuJoCo env; IMU modes in config	**Completed:** training/eval pipeline and archived runs; hardware policy deployment not fully exercised here
III	Visual or pose features f_t; distance-centric rewards	**Not started:** specification and literature basis only

6 Technical Discussion

6.1 Biological and control context

Decentralized feedback atop rhythmic coordination remains a standard idiom for insect locomotion [4, 5]. Attitude filters for accelerometer–gyro fusion [6, 8] and posture control on terrain [7] inform the IMU observation choices in simulation. Disturbance-triggered transitions [9] motivate reflex layers in the design spec, not a completed hardware demo in this capstone.

6.2 Planned adaptive layer (design specification)

Internal architecture notes (`docs/ARCHITECTURE.md`) describe a **future** two-timescale stack: a **fast tick** ($\sim$10–20 ms) for IMU read, attitude fusion, stability score S, reflex adjustments, gait interpolation, and servo commands; and a **slow tick** (per gait cycle) for windowed costs and optional parameter adaptation. That design informed IMU-inclusive observation modes in simulation; it is **not** claimed as deployed firmware here. The complementary blend

$$\alpha \in (0,1), \quad \theta_{\text{roll}} \leftarrow \alpha(\theta_{\text{roll}} + \omega_x \Delta t) + (1-\alpha)\,\theta_{\text{roll,accel}}, \tag{12}$$

(and analogously for pitch) is the nominal attitude update.

6.3 Open-loop sinusoidal gait (implemented)

For the no-IMU milestone, each leg follows resting abduction and flexion angles plus sinusoidal variation at common frequency f, with tripod group B offset by π from group A. This is the same law optimized in Sec. 2.1 and delivered in Track I (Sec. 3). Amplitudes follow default tables shared by Python and `optimized_gait_params.h`.

6.4 MuJoCo and reinforcement learning (simulation)

High-fidelity training uses Gymnasium + MuJoCo [1] with PPO [2] as implemented in Stable-Baselines3 [3]; the mathematical objective is Eq. (9) with advantages (7). Tracks II–III (Sec. 4–5) specialize the observation and reward; YAML configs set γ, λ, learning rate, network widths, PD gains, and terrain. A representative static render of the simulation model appears in Fig. 4 (Sec. 9.2).

6.5 Mechanical design: CAD and 3D-printed parts

Walter's chassis and legs are built from **parts designed in CAD** and produced by **additive manufacturing** (3D printing): a central body, six leg assemblies, and two-degree-of-freedom linkages per leg (abduction and flexion axes) that mate to the servo output splines. The design prioritizes a repeatable assembly for twelve actuated joints, clearance for wiring, and attachment of the electronics tray and battery. Figure 1 shows an overall render; Figure 2 shows the body, leg, and linkage components used in the assembly. Fasteners and servo hardware complete the mechanical stack; material usage (filament mass) and printed-part cost are not included in the dollar totals in Tables 3–5.

6.6 Electronics, Hexapod V2 PCB, and safety

The bill of materials matches purchased components: Teensy-class MCU (SparkFun 16771), Pololu 1354 eighteen-channel driver, buck converter, BMI270 via Qwiic cabling, dual-cell LiPo and charger, terminals, and servos. Custom *Hexapod V2* PCBs (revision v5 dated 2026-04-03 on the latest order) integrate power and signal routing for Walter's stack; Figure 3 shows a board photo and schematic excerpt. `docs/FIRMWARE.md` records UART to the servo controller, channel ordering FL/FR/ML/MR/RL/RR with abduction then flexion per leg, and pulse mapping $u_k = \mathrm{clamp}(c_k + d_k q_k)$ (typical bounds 600–2400 μs). Open-loop firmware applies degree-to-command gains and clamps before streaming compact-protocol targets; these bench gains are not outputs of the reduced-order optimizer.

7 Requirements and Constraints

7.1 Source of requirements

Engineering requirements are traced to phase exit criteria in `docs/PHASES.md` (Phases 1–4: IMU telemetry, reflexes, cautious mode and recovery, online adaptation), supplemented by safety and interface constraints in `docs/FIRMWARE.md` and the open-loop scope in `firmware/hexapod_openloop_sine/README.m`
`PHASES.md` serves as the team's consolidated requirements-and-constraints specification for firmware behavior.

7.2 Summary: met vs. not met

Satisfied in this project: discrete tripod baseline firmware; open-loop optimization with JSON export and header sync; calibration sketches; CAD-documented mechanical design (Figs. 1–2); a MuJoCo PPO workflow with configs, training scripts, checkpoints, and evaluation writeups. **Not satisfied against the internal phased firmware spec:** Phases 1–4 adaptive behaviors (telemetry-only through online adaptation) are largely **not implemented** on the MCU in the snapshot summarized below.

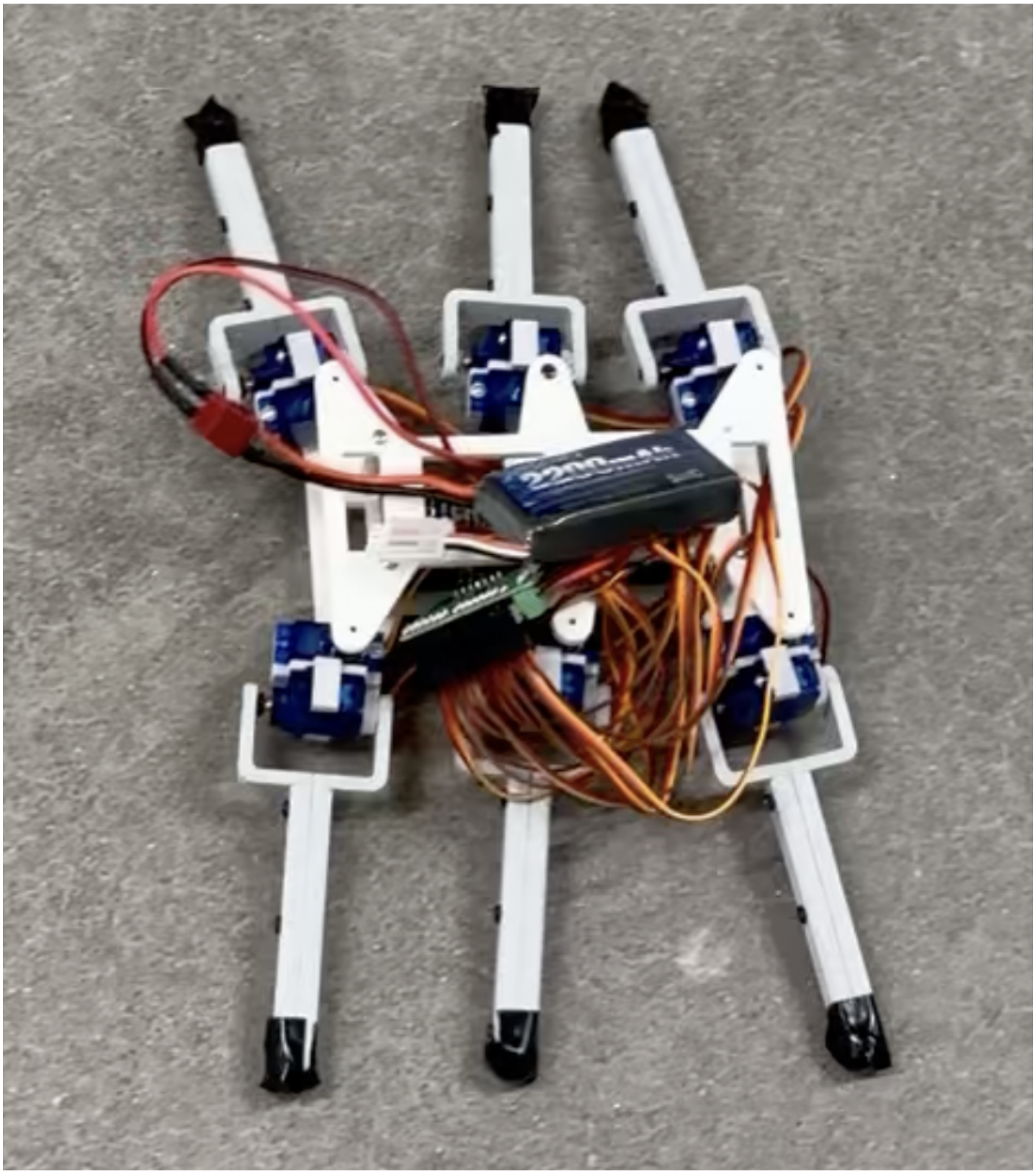
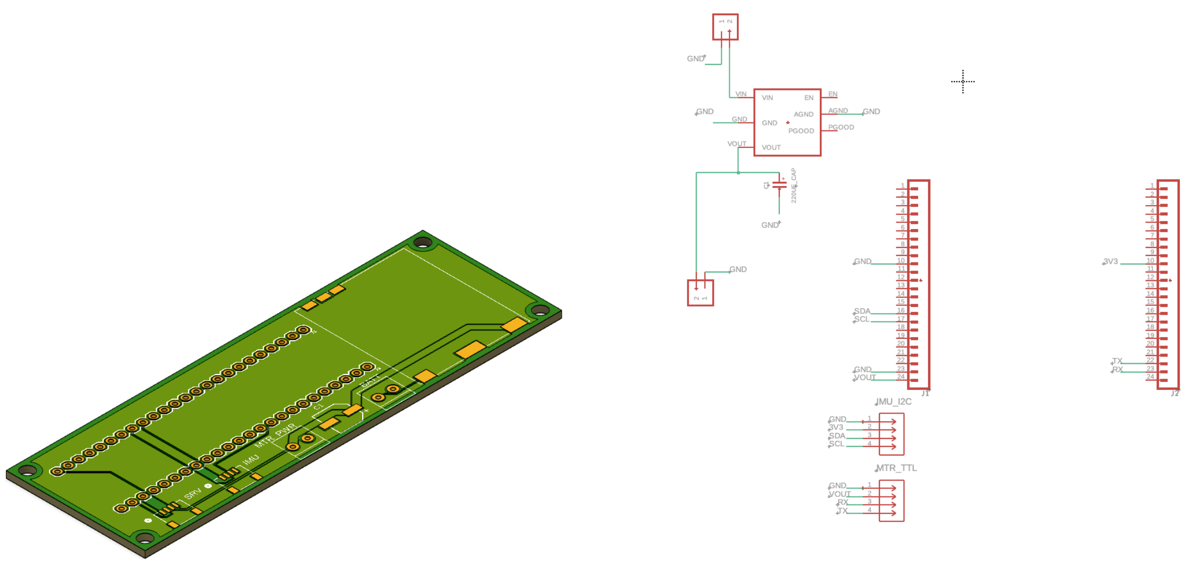


Figure 1: Walter: full-assembly render of the twelve-DOF hexapod (CAD; Walking Alternating Tripod for Evolutionary Research).

(a) Body (b) Leg assemblies (c) Linkage

Figure 2: CAD-designed 3D-printed subassemblies for Walter: chassis, legs, and per-leg linkage.



(a) Assembled PCB (Hexapod V2) (b) Schematic (excerpt)

Figure 3: Custom electronics for Walter: Hexapod V2 board and schematic.

Table 2: Requirement traceability (internal PHASES.md vs. delivered work).

ID	Requirement	Status	Evidence / notes
R1	Phase 1: IMU read, complementary filter, USB telemetry ∼50 Hz; gait unchanged	Not met (in repo)	PHASES.md; folder may be missing
R2	Phase 2: postural reflexes, clamps, serial tuning, EEPROM	Not met (in repo)	ARCHITECTURE.md pseudocode
R3	Phase 3: stability score, cautious mode, push recovery, fall freeze	Not met (in repo)	Documented only
R4	Phase 4: five-parameter adaptation, cost J, SPSA/bandit, EEPROM	Not met (in repo)	Documented only
R5	Hardware: MCU + eighteen-channel driver + servos + documented map + PCBs + Walter mechanical CAD/3D parts	Met	BOM, Figs. 1–3, FIRMWARE.md
R6	Safety: pulse clamps; parameter slew (adaptive layer)	Partial	Clamps in v2/openloop; slew in spec
R7	Open-loop: optimizer/firmware parity; JSON export	Met	Optimizer, sync_gait_params_from_json.py
R8	Simulation: MuJoCo env, PPO training, documented evals	Met	hexapod_training_new/, e90_repo/released_checkpoints/

7.3 Project cost

Tables 3–4 summarize **purchased** electronics (three identical platforms) and **fabrication** costs for three PCB/stencil orders. Table 5 states the **grand total** for those categories only; filament, fasteners, and shop labor are outside the accounting. Servo counts in Table 3 are per platform (eight units listed per BOM line); completing twelve DOF per robot may require additional servos beyond those lines if not already stocked.

8 Methods

8.1 Mechanical assembly (*Walter*)

Printed body and leg components are fastened per the CAD design; each leg receives two servos driving abduction and flexion through the linkage. The custom PCB mounts to the chassis and connects to the Pololu driver and Teensy; power and ground respect the buck converter and screw terminals from the BOM. Bench alignment uses `set_zero` and side-only test sketches before closed-loop or open-loop gait tests.

Table 3: Electronics bill of materials, **one platform** (USD). The team purchased three identical sets.

Part	Description	Vendor	Qty	Unit	Line
SparkFun 16771	Microcontroller	DigiKey	1	37.20	37.20
Pololu 1354	18-channel servo driver	DigiKey	1	46.95	46.95
DFRobot DFR1202	Step-down buck converter	DFRobot	1	12.90	12.90
SparkFun BMI270	6-DoF IMU	DigiKey	1	18.50	18.50
SparkFun 11855	2×7.4 V LiPo + charger	Amazon	1	27.99	27.99
SparkFun 14417	Female Qwiic connector	DigiKey	2	0.60	1.20
SparkFun 17259	100 mm Qwiic cable	DigiKey	1	1.95	1.95
SparkFun 17261	Qwiic cable to female header	DigiKey	1	1.95	1.95
Amphenol YO0201500000G	1×2 screw-down terminal	DigiKey	2	1.28	2.56
DFRobot SER0039	Servo motor	DigiKey	8	5.90	47.20
Subtotal	**one platform**				**198.40**
Electronics total	**three platforms (×3)**				**595.20**

Table 4: PCB and stencil orders, *Hexapod V2* (USD order totals including merchandise, shipping, duties/taxes, and sales tax where charged by the vendor).

Order ID	Description / notes	Status	Total
Y4-11911448A	PCB prototype (5 pcs) + SMT stencil; v5_2026-04-03	Awaiting payment	74.24
W2026030804205410	PCB + stencil shipment	Shipped	68.26
W2026021709554129	PCB + stencil (earlier order)	Shipped	54.97
PCB subtotal			**197.47**

Table 5: Grand total project cost (USD) for reported purchases and fabrication.

Category	Amount
Electronics (three platforms)	595.20
PCB fabrication and stencils (three orders)	197.47
Grand total	**792.67**

8.2 Baseline discrete gait

`firmware/hexapod_walking_v2/hexapod_walking_v2.ino` implements a deterministic tripod state machine with smoothed keyframes (`moveSmoothTo`).

8.3 Reduced-order simulation optimization

`hexapod_brain_roadmap copy/ode_optimization_no_imu/hexapod_no_imu_optimization.py` integrates the surrogate model with `scipy.integrate.solve_ivp`, evaluates the objective on a steady-state window, and calls `scipy.optimize.minimize` (L-BFGS-B). Outputs include `no_imu_optimization_s`
and optional CSV/plots.

8.4 Parameter sync and open-loop firmware

From `firmware/hexapod_openloop_sine/`, run `python3 sync_gait_params_from_json.py` to regenerate `optimized_gait_params.h`. The sketch evaluates the sinusoidal law at `INTERP_DELAY_MS` (default 10 ms), applies startup frequency ramping, maps degrees to integer commands, clamps, and sends Pololu compact-protocol targets on `Serial1`.

8.5 Bench testing and calibration

`firmware/set_zero/` and side-only test sketches support calibration. The open-loop README documents sign and scale tuning when feet drag or the robot shuffles backward.

8.6 MuJoCo PPO training and evaluation

The project produced a **reusable training workflow**: `train.py` reads YAML from `configs/`, writes summaries, checkpoints, and logs under runs/<run_name>/, and pairs with `evaluate.py` using the same configuration. Released material includes zipped policies, per-run evaluation folders with `REPORT.md` summaries, and metadata for reproducibility and reporting.

9 Results

9.1 Reduced-order optimization (surrogate model)

Table 6 summarizes `hexapod_brain_roadmap copy/ode_optimization_no_imu/no_imu_optimization_summary`
These numbers describe the *planar surrogate* only; they are not a guarantee of on-robot speed or stability.

Table 6: Optimizer-reported metrics (committed summary JSON).

Quantity	Value
Gait frequency f	1.80 Hz
Mean forward speed (steady window)	0.234 m/s
Total displacement (rollout)	1.86 m
Roll RMS	0.440 rad
Pitch RMS	~ 0 (near zero in file)
Vertical position RMS	0.086 m
Peak servo usage ratio (proxy)	0.152
Contact balance std. dev.	0.388
Dominant oscillation frequency (estimated)	3.54 Hz
Objective cost $\mathcal{J}(\boldsymbol{\phi}^{\star})$	-228.38

Reading Table 6. Mean forward speed ≈ 0.23 m/s and ≈ 1.9 m travel over the surrogate horizon indicate the optimizer found a gait that moves the reduced model forward. Roll RMS near 0.44 rad is **large**: the surrogate tolerates body oscillation in exchange for speed under the chosen weights. Pitch RMS near zero reflects the planar symmetry of the lumped model more than a claim of perfect hardware pitch stability. Peak servo usage and contact-balance spread show the solution stays inside soft actuator and support proxies, but **contact and torque models are coarse**—hardware will differ once pulses, friction, and foot slip are included.

Table 7 lists $\boldsymbol{\phi}^{\star}$ exported to `optimized_gait_params.h`. Flexion phase shifts sit at the lower bound ($-90°$): the optimizer traded stride phasing aggressively for the surrogate objective; bench tuning may move off that bound. Rest angles cluster in physically plausible ranges; frequency near 1.8 Hz matches a moderate stride rate for small hexapods in simulation-like settings.

Table 7: Optimized decision variables $\boldsymbol{\phi}^{\star}$ from L-BFGS-B (same JSON as Table 6). Angles in degrees; leg order FL, FR, ML, MR, RL, RR.

Quantity	Value
Gait frequency f	1.800 Hz
Abduction rest $\boldsymbol{\theta}_{\text{abd,rest}}$	(10, 10, 0, 0, -10, -10)
Flexion rest $\boldsymbol{\theta}_{\text{flx,rest}}$	(32.00, 32.00, 34.00, 34.00, 32.00, 32.00)
Abduction phase $\boldsymbol{\delta}_{\text{abd}}$	$\approx \mathbf{0}$ (magnitude $< 10^{-6}$ deg)
Flexion phase $\boldsymbol{\delta}_{\text{flx}}$	(-90, -90, -90, -90, -90, -90)
Swing lift amplitude	12.0 deg (`lift_swing_deg_peak`)

9.2 MuJoCo simulation (high-fidelity model)

Track II–III training uses Gymnasium with the MuJoCo asset `hexapod_training_new/assets/hexapod.xml`: checkerboard ground, twelve torque-controlled joints, and RK4 integration at 0.002 s (see configs for frame skipping). Figure 4 is an **offscreen MuJoCo render** (software `Renderer`, no interactive viewer): the model was advanced under gravity with zero actuator commands until the body settled on the plane, then a fixed camera was used. The view documents the mesh, materials, and lighting used for PPO rollouts; it is not a policy-controlled frame. Interactive inspection is available via `scripts/live_mujoco_viewer.py` with a trained checkpoint.

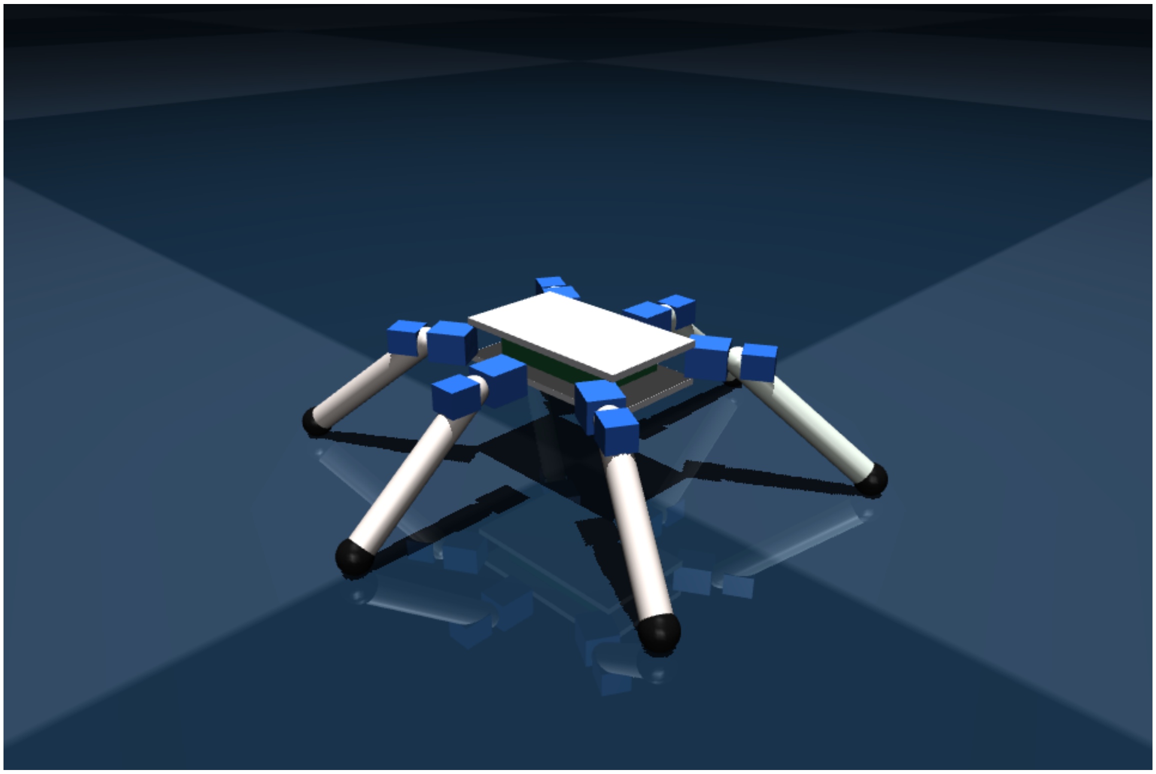


Figure 4: MuJoCo screenshot of the Walter hexapod model used for PPO training and evaluation (post-settle, zero control; offscreen render).

Figure 4 confirms the asset geometry and contact setup used for learning; it is a passive settle, not evidence of policy quality. Training and evaluation videos for specific checkpoints provide the appropriate visual readout of learned behavior.

9.3 MuJoCo PPO evaluations (released checkpoints)

Table 8 quotes short evaluation protocols documented under `e90_repo/released_checkpoints/evaluations/`. Rewards are **not** comparable across configs (different shaping and scaling); forward displacement and path length provide a common physical readout within each experiment family.

Reading Table 8. The rows are **not** comparable by raw reward (shaping and scaling differ). Forward displacement and path length are coarse physical readouts: the circle-path MLP run shows modest net forward motion and path length consistent with a curved task; the terrain run averages shorter forward progress when steps and roughness dominate. These figures support the claim that **policies were trained and evaluated in MuJoCo**, not that a single "best" locomotion score was achieved across tasks. Treat simulation displacements as **indicative**, not predictive of

Table 8: Representative PPO evaluation means (300 env steps per episode unless noted; see each REPORT.md).

Checkpoint	Task / policy	Fwd. $+x$	2D path	Notes
mlp_clean_run005_best	Circle path; MLP 256×256 PPO; training run mlp_run_005 (301k steps).	0.18 m	0.27 m	Mean reward ≈ 6800.
terrain_low_shake_run001_best	Flat, rough, and step terrain; MLP PPO.	0.33 m (3 ep. mean)	—	Flat/rough ≈ 0.47 m forward; steps harder.

hardware distance without deployment tests.

The `pd_velocity_window_run018_best` checkpoint is an sCPG + PD + velocity-window experiment with small net displacement in its released short eval—useful as an archival structured-policy datapoint, not a headline locomotion result (see that folder's `REPORT.md`).

10 Discussion

What this capstone demonstrates. The team built a **named hexapod platform** with documented mechanics, electronics, and cost; implemented an **optimization-to-firmware** path with reproducible parameters; and ran a **MuJoCo PPO training and evaluation** workflow with archived checkpoints and configs. Together, that is a credible **platform-and-methods** outcome: reusable hardware, a defensible open-loop pipeline, and simulation evidence for learned locomotion behaviors under stated rewards.

Why full adaptive locomotion is not claimed. Closing adaptive loops on hardware requires time for IMU bring-up, reflex tuning, safety testing, and power-aware deployment. The phased firmware requirements in `PHASES.md` were **not** met in the repository snapshot (Table 2); stating that once in requirements and here avoids repeating it in every technical subsection.

Technical bottlenecks. Surrogate versus MuJoCo versus hardware disagree on contact, friction, and servo dynamics; manual degree-to-pulse mapping separates optimized angles from reliable motion; open-loop gaits do not reject disturbances. RL rewards are task-specific, so **cross-comparing raw returns** across YAML files is misleading—compare displacements and behaviors within a task family only, with humility about sim-to-real.

Sim claims. MuJoCo results show that policies *can* be trained and evaluated under the project's models and rewards. They are **not** automatic predictors of field performance without transfer experiments.

Credible next steps. Harden IMU read paths and logging on the Teensy; pick one PPO checkpoint and measure a short hardware rollout with matched observation rate; then iterate reflexes or residuals before adding vision or distance-only objectives.

Advice for future teams. Lock a baseline gait and logging format before layering sensors; budget bench time after every simulation milestone; track PCB revisions and BOM multiples from week one.

11 Conclusion

This capstone **established** *Walter*: a twelve-DOF hexapod with CAD/3D-printed structure, custom PCBs, and purchased electronics totaling US$792.67 in tracked spending. The team **completed** an open-loop gait optimization loop with deterministic export to firmware, and **completed** a MuJoCo PPO training and evaluation toolchain with stored checkpoints and representative evaluations. A **bio-inspired, layered adaptive** control story informed design and simulation choices but was **not** fully realized on the MCU; that gap is documented in requirements traceability (Table 2) rather than hidden. The project provides a **strong base** for future adaptive and sim-to-real work; code and this report are maintained at `https://github.com/HowardWHSrun/WALTER`.

Acknowledgements

We thank our advisor, Allan Moser, for guidance on the project. We thank Swarthmore Engineering and shop or lab staff who supported fabrication and testing. No funding sources beyond the Engineering department are declared for this accounting.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 5026–5033, 2012.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations." Journal of Machine Learning Research, 2021.

[4] Y. O. Aydin *et al.*, "Mechanosensory control of locomotion in animals and robots: Moving forward," *Biomimetics*, vol. 8, no. 4, p. 211, 2023. PMC10445419.

[5] G. Chen *et al.*, "Adaptive locomotion control of a hexapod robot via bio-inspired learning," *Frontiers in Neurorobotics*, vol. 15, p. 627157, 2021.

[6] R. G. Valenti, I. Dryanovski, and J. Xiao, "Keeping a good attitude: A quaternion-based orientation filter for IMUs and MARGs," *Sensors*, vol. 15, no. 8, pp. 19302–19330, 2015.

[7] L. Bai *et al.*, "Development and implementation of a new approach for posture control of a hexapod robot to walk in irregular terrains," *Robotica*, vol. 37, no. 12, pp. 2225–2238, 2019.

[8] R. Zhang *et al.*, "Attitude estimation algorithm of portable mobile robot based on complementary filter," *Micromachines*, vol. 12, no. 11, p. 1373, 2021.

[9] A. Johnson, K. D. Gregg, and D. E. Koditschek, "Disturbance detection, identification, and recovery by gait transition in legged robots," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 3627–3634, 2010.

A Public repository

The public project URL is `https://github.com/HowardWHSrun/WALTER`. Appendix paths below refer to the local Engineering 90 workspace layout at the time of writing; the GitHub repository is intended to subsume or mirror these components for reproducibility.

B Repository map (selected local paths)

- **Adaptive firmware and docs:** `RL Temp/adaptive_ondevice_hexapod/README.md`, `docs/DESIGN.md`, `docs/ARCHITECTURE.md`, `docs/PHASES.md`, `docs/FIRMWARE.md`
- **Firmware sketches:** `firmware/hexapod_walking_v2/`, `firmware/hexapod_openloop_sine/`, `firmware/set_zero/`, side-only tests
- **Reduced-order optimization:** `hexapod_brain_roadmap copy/ode_optimization_no_imu/`
- **MuJoCo PPO training:** `RL Temp/hexapod_training_new/` (`train.py`, `configs/`, `runs/`)
- **Released checkpoints and evals:** `RL Temp/e90_repo/released_checkpoints/`
- **Presentation notes:** `Presentation/E90_MidSemester_SpeakerScript.tex` (and PDF)
- **This folder (figures):** `walter_assembly.png`, `Body.png`, `Legs.png`, `Linkage.png`, `pcb_board.png`, `pcb_schematic.png`, `mujoco_hexapod_sim.png` (plus originals with longer filenames)